JETBRAINS | Kotlin

The Java Virtual Machine & the Kotlin Compiler

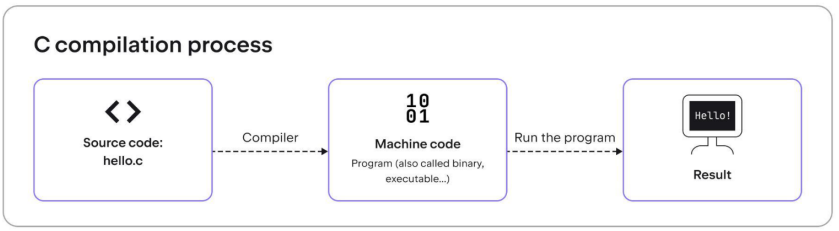
kotlinglang.org/education



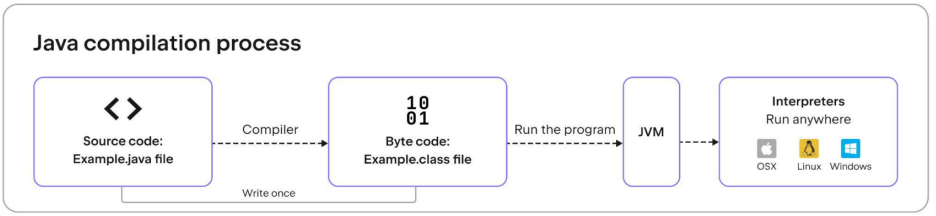
The Java language

- Was created in 1995.
- Is an OOP language with strong static typing.
- Has Just-in-time (JIT) compilation.
- Uses the Java Virtual Machine (JVM).
- Has a garbage collector, meaning you can allocate memory and it will be freed automatically.

Compilation process – Java vs C



VS



Java bytecode

```java
public class Main {
    public static void main(String[] args) {
        System.out.print("Hello, World!");
    }
}
```

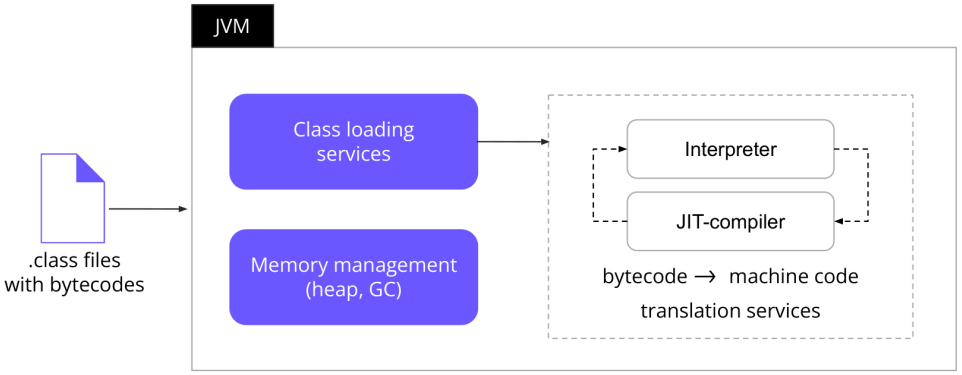


```
public class examples/Main {


  public <init>()V
   L0
    LINENUMBER 3 L0
    ALOAD 0
    INVOKESPECIAL java/lang/Object.<init> ()V
    RETURN
   L1
    LOCALVARIABLE this Lorg/examples/Main; L0 L1 0
    MAXSTACK = 1
    MAXLOCALS = 1


  public static main([Ljava/lang/String;)V
   L0
    LINENUMBER 5 L0
    GETSTATIC java/lang/System.out : Ljava/io/PrintStream;
    LDC "Hello, World!"
    ICONST_0
    ANEWARRAY java/lang/Object
    INVOKEVIRTUAL java/io/PrintStream.print
           (Ljava/lang/String;[Ljava/lang/Object;)Ljava/io/PrintStream;
    POP
   L1
    LINENUMBER 6 L1
    RETURN
   L2
    LOCALVARIABLE args [Ljava/lang/String; L0 L2 0
    MAXSTACK = 3
    MAXLOCALS = 1
}
```

The JVM under the hood



Memory organisation

JVM memory is divided into two parts:

-
-

Static memory, or non-heap memory, is created when the JVM starts, and it primarily stores class structures, fields, method data, and the code for methods and constructors.

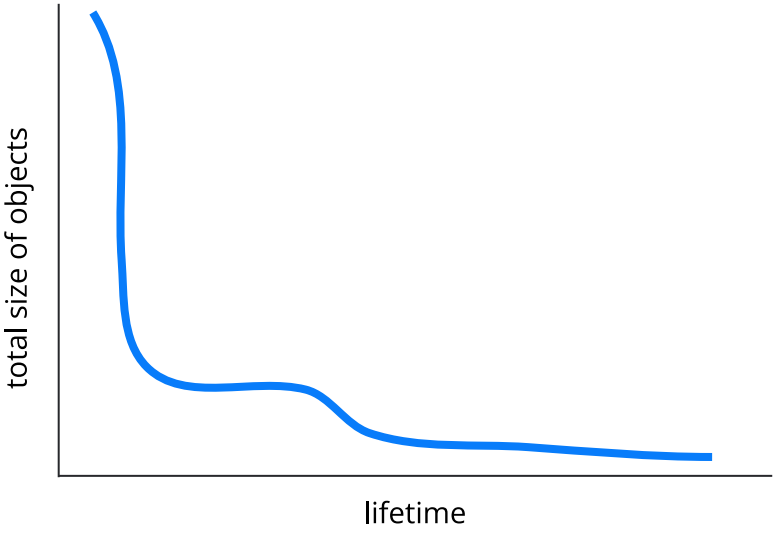
  - Loaded classes
  - Stacks of all threads
  - Service memory for the JVM itself
  - Small – roughly 1024 KB for stacks

Heap memory is the runtime data area shared among all JVM threads, and it is used to allocate memory for all Java objects.

  - All objects created during a program's execution

  - Large – from several MB up to several TB

Weak generational hypothesis



According to the weak generational hypothesis, objects tend to die young.

A related assumption is that, as an object lives longer, the likelihood will be that it will continue to live increases.

Generations

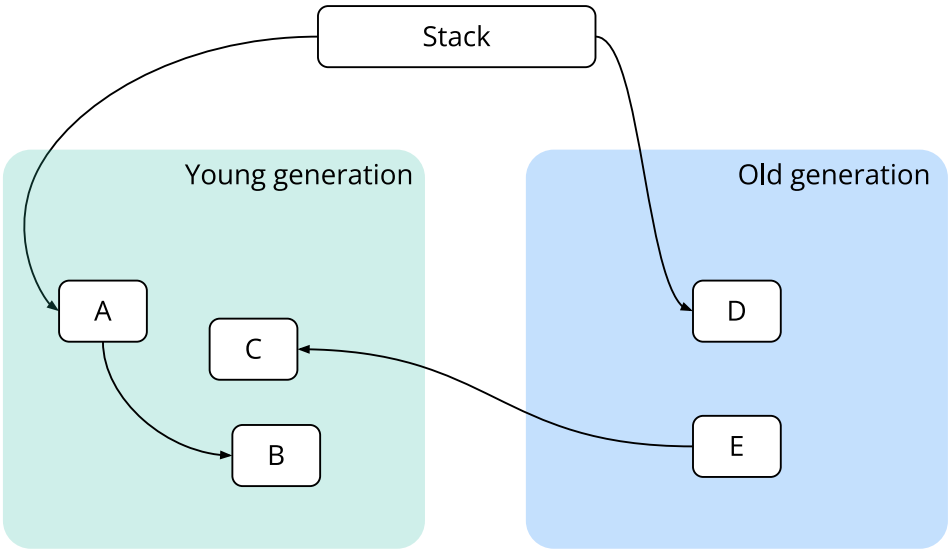
The memory of a program can be divided into two generations:

- Young
- Old

Garbage collection (GC) can be similarly divided:

- Small GC clears only objects from young generation
- Full GC clears objects from both generations

Dead objects



Serial garbage collector

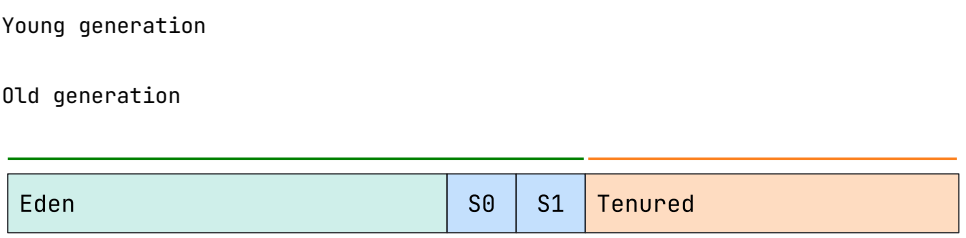
The first garbage collector created was the serial garbage collector, which is single-threaded, and the parallel and CMS garbage collectors are based on it.

-

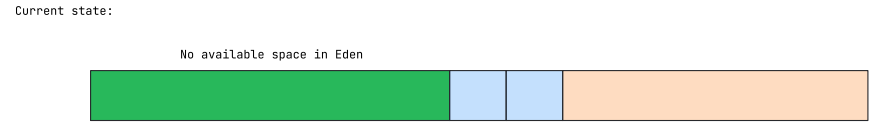
In the serial garbage collector, the heap is divided into 4 areas:

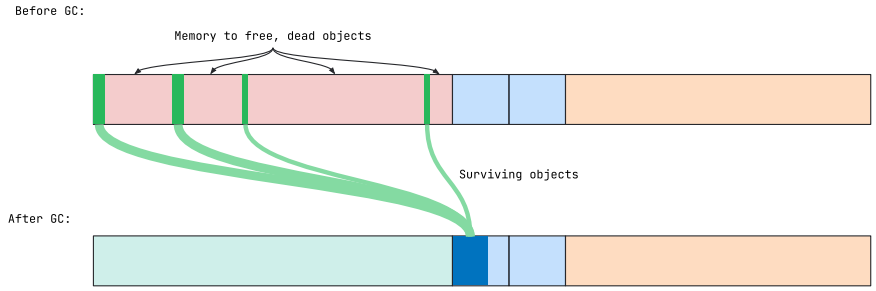
  - Eden – Roughly 8/10 of the young generation
  - Survivor 0 – Roughly 1/10 of the young generation
  - Survivor 1 – Roughly 1/10 of the young generation
  - Tenured – Roughly 2/3 of the heap

- (Almost) All objects are created in the Eden area.

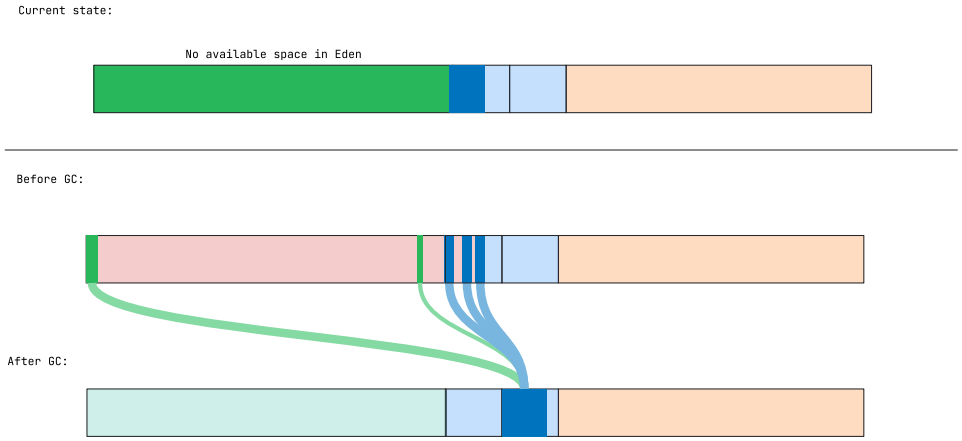


How garbage collection works

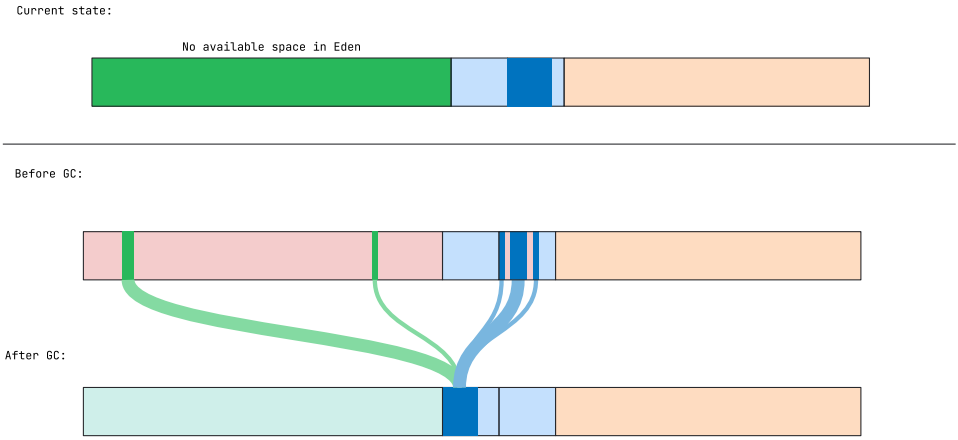




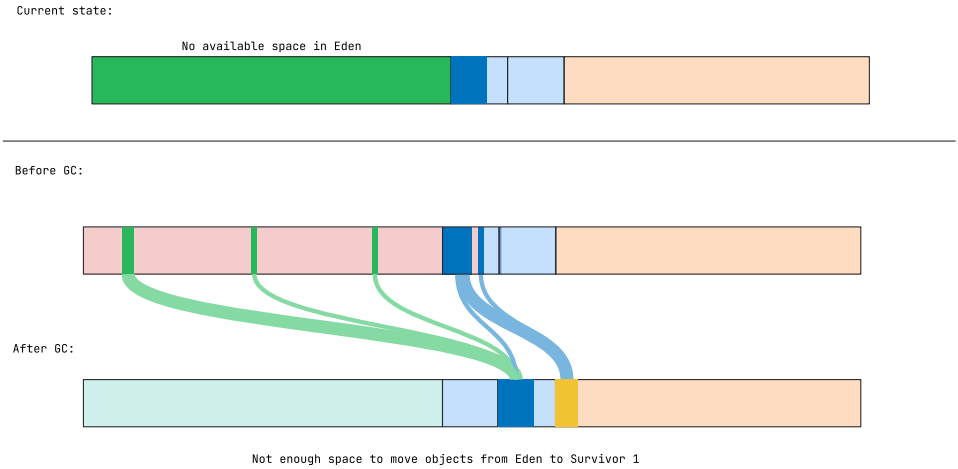
How garbage collection works



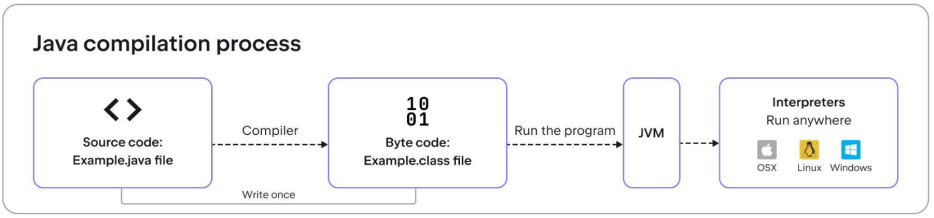
How garbage collection works



How garbage collection works



Just-in-time compilation



- Program profiling occurs at runtime.
- Pieces of code are compiled for a specific platform to optimize the execution time.

Interpreting a command is much slower than executing it directly on the processor.



Why, then, do we need the interpreter?

Just-in-time compilation



Why, then, do we need the interpreter?

Interpreter

- Starts working almost instantly.
- The performance of the executable code is poor.

VS

JIT-compiler

- Kicks in after a long delay (needs time for optimizations).
- The performance of the executable (compiled) code is high.

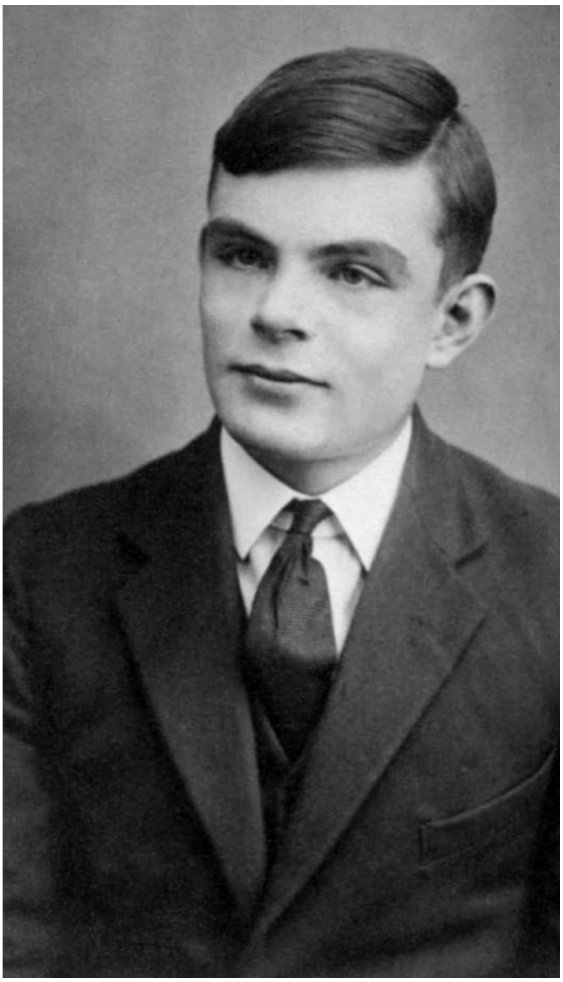
Just-in-time compilation



What sorts of JIT code are worth compiling?

Code that will take a long time to run or code that runs frequently, because the compilation overhead will be covered by the profit from having optimized execution.

Just-in-time compilation



How can we understand which pieces of code will take a long time to execute?

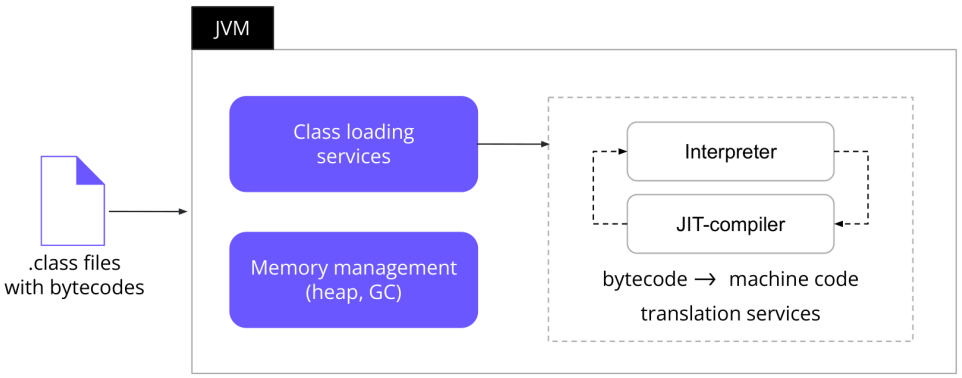
Some guy named Alan Turing said it is **impossible**.

But empirically it is possible. If a piece of code took a long enough time to execute once, then most likely the same will be true in the future.

The JVM under the hood



Why does the compiler need to access the interpreter?



The JVM under the hood



Why does the compiler need to access the interpreter?

```
class PiUtils {
    private static final double PI = 3.141592653589;

    public static double getPiSquared() {
        return PI * PI;
    }
}
```



```
public static double getPiSquared() {
    return 9.869604401084375;
}
```

The JVM under the hood



Why does the compiler need to access the interpreter?

```
class PiUtils {
    private static final double PI = 4;

    public static double getPiSquared() {
        return PI * PI;
    }
}
```



Reflection

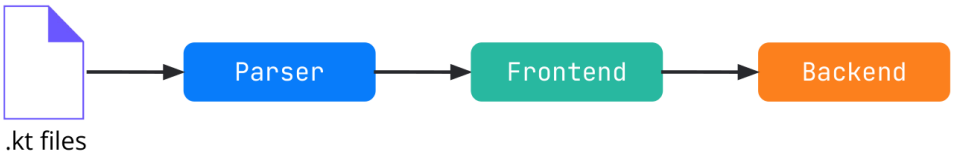
```
public static double getPiSquared() {
    return 9.869604401084375;
}
```



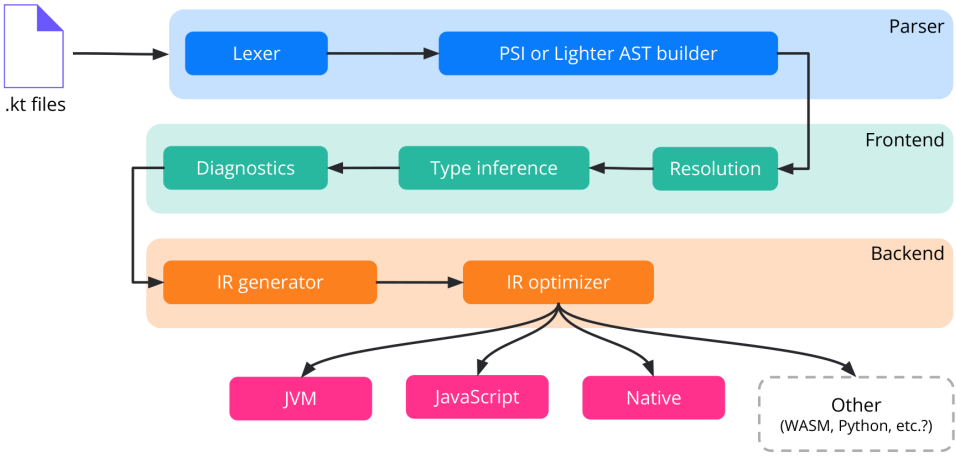
The Kotlin language

- Can be compiled into JVM bytecode.
- Is interoperable with Java.
  - Can work in Java projects.
  - Can use Java libraries.
- Is safe and concise.
- Provides the ability to integrate into the compilation process (compiler plugins).
- Is the main development language for Android applications, according to Google.

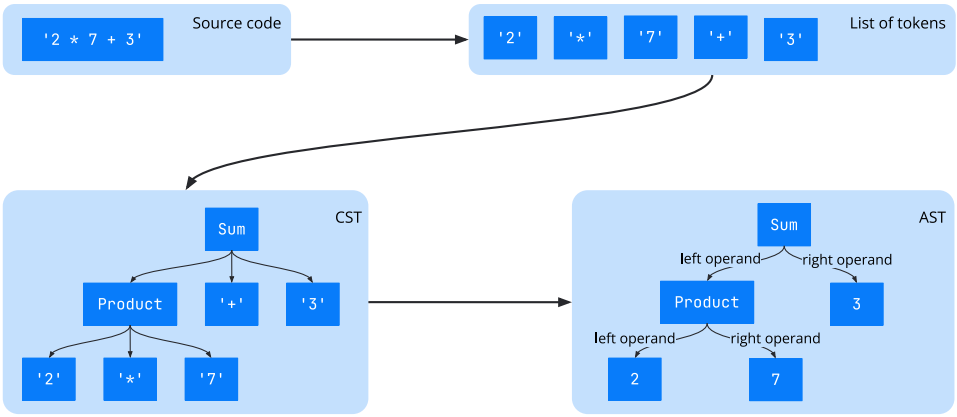
Kotlin compiler



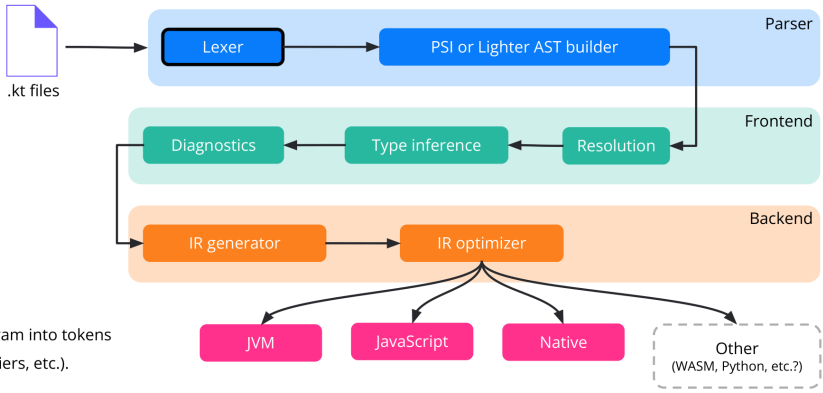
Kotlin compiler



Parsing in a nutshell

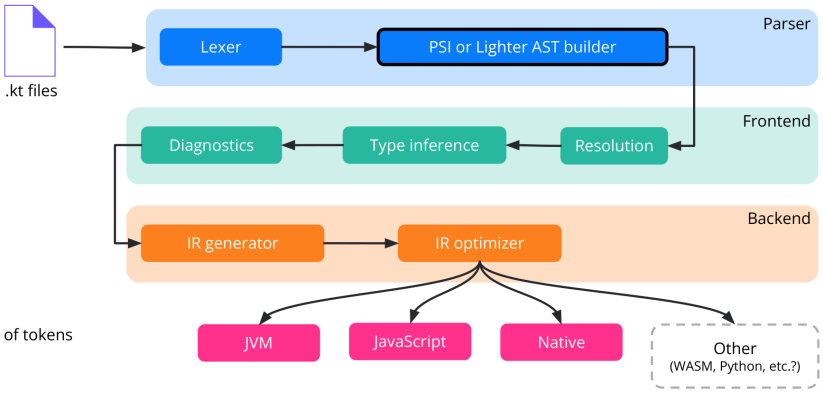


Kotlin compiler



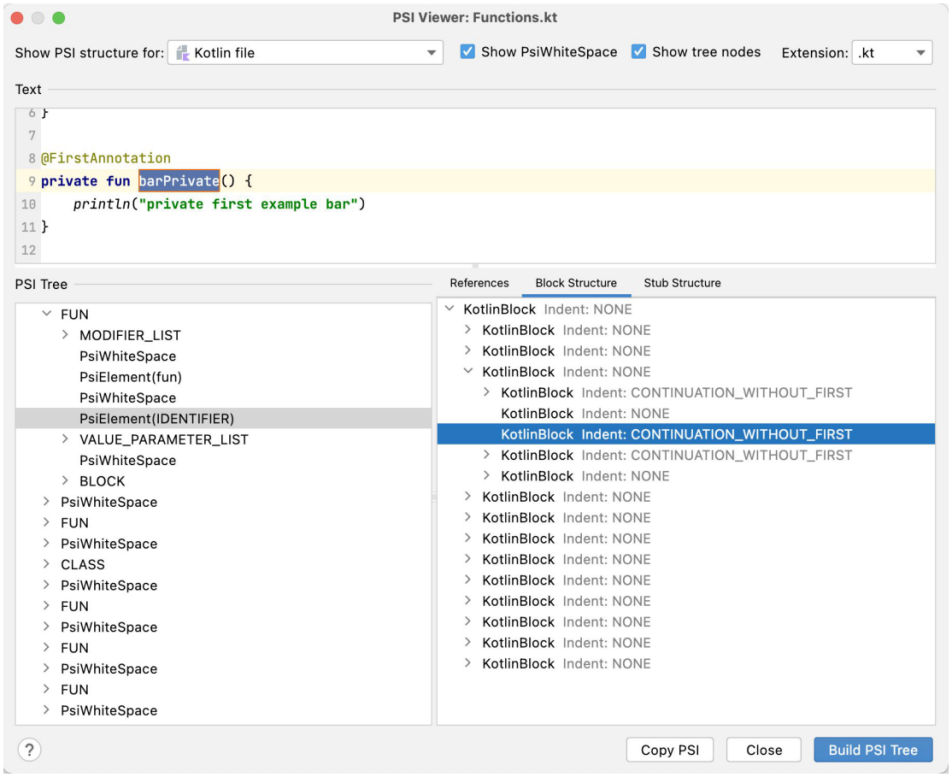
Splitting the program into tokens (keywords, identifiers, etc.).

Kotlin compiler

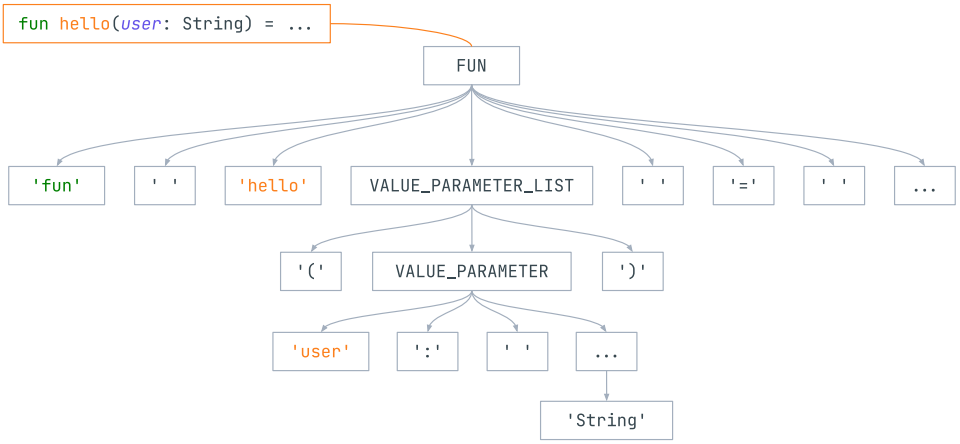


Converting the list of tokens into CST or AST

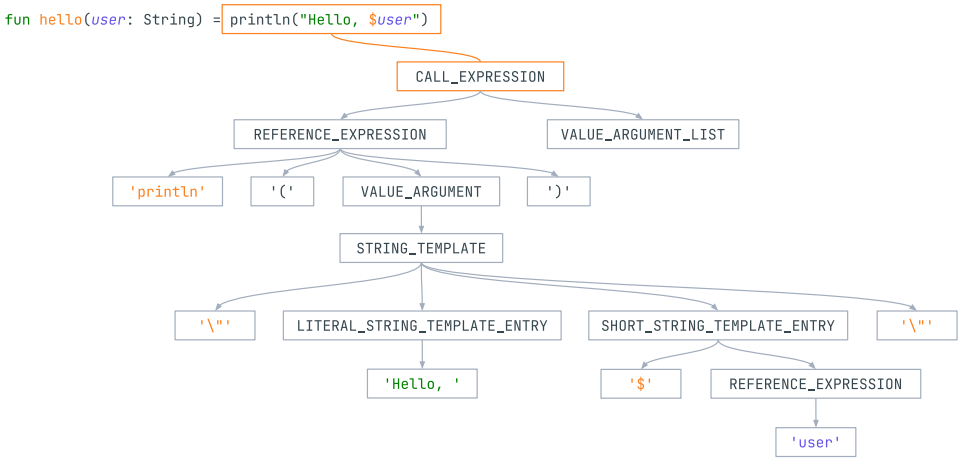
Kotlin compiler: PSI



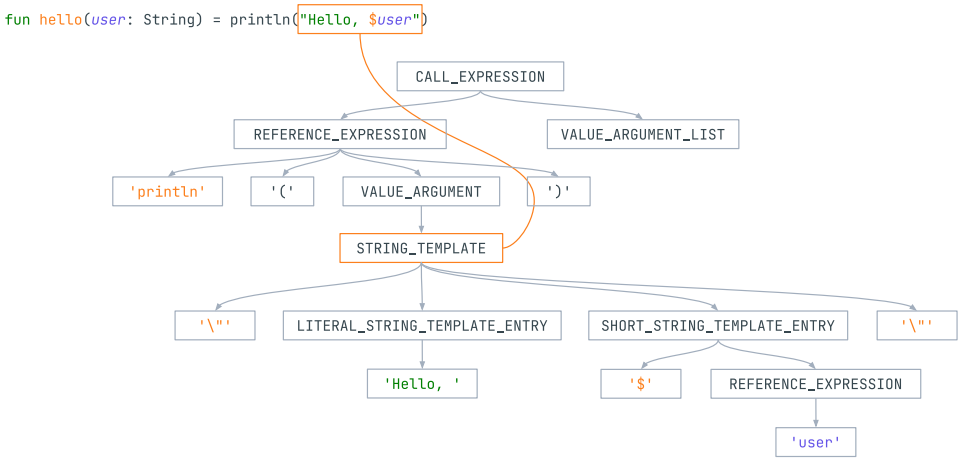
Kotlin compiler: PSI



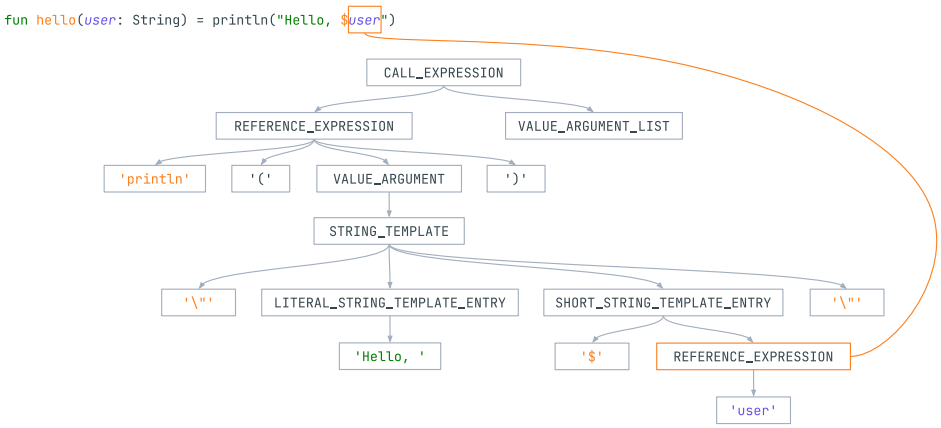
Kotlin compiler: PSI



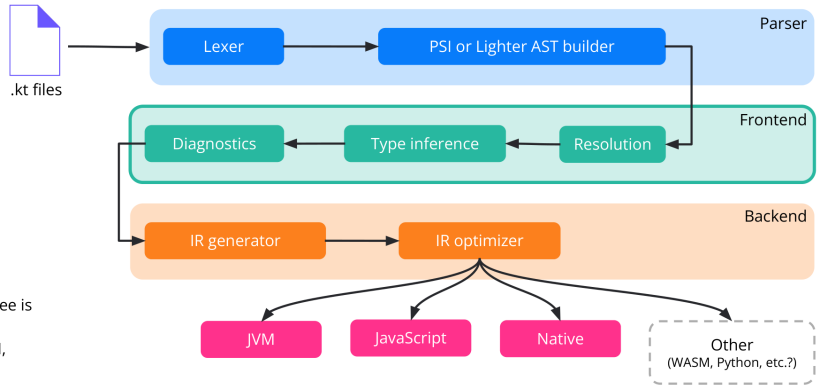
Kotlin compiler: PSI



Kotlin compiler: PSI

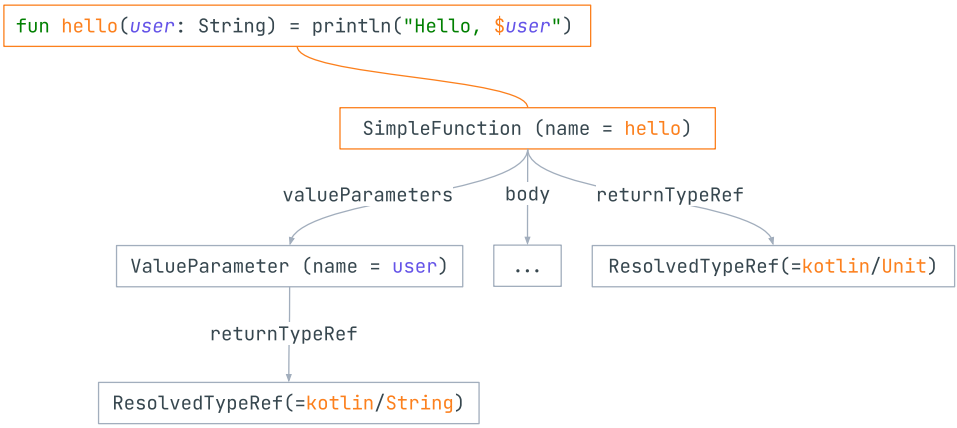


Kotlin compiler

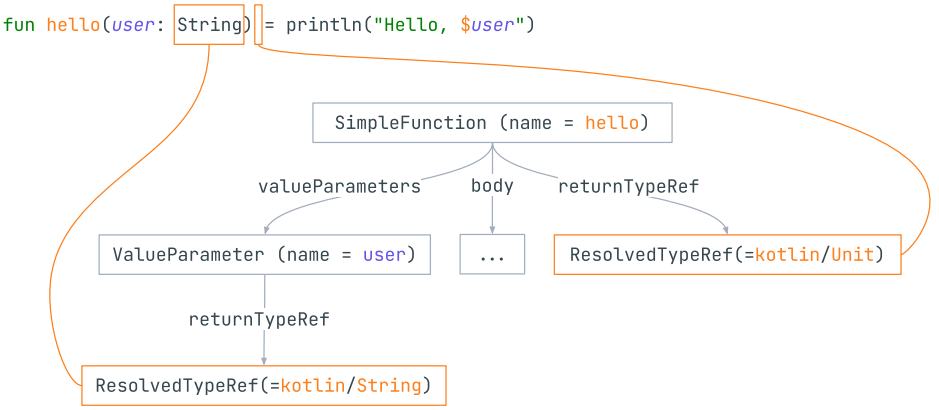


The so-called FIR tree is being built,. It's an analogue of the PSI, but it's mutable.

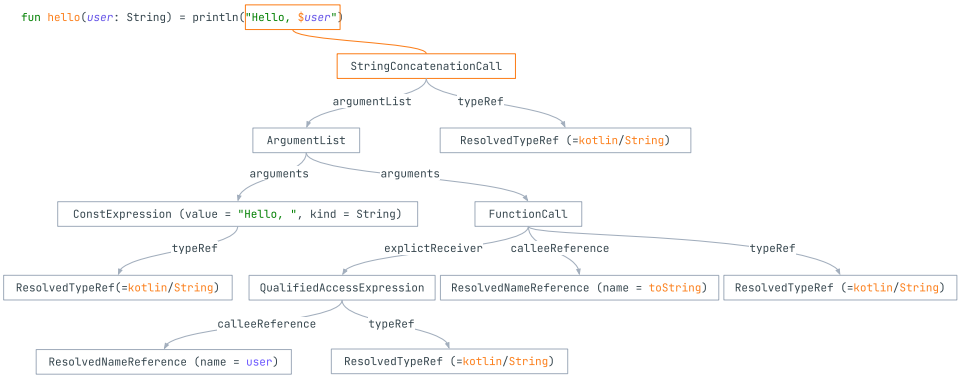
FIR? Another tree!



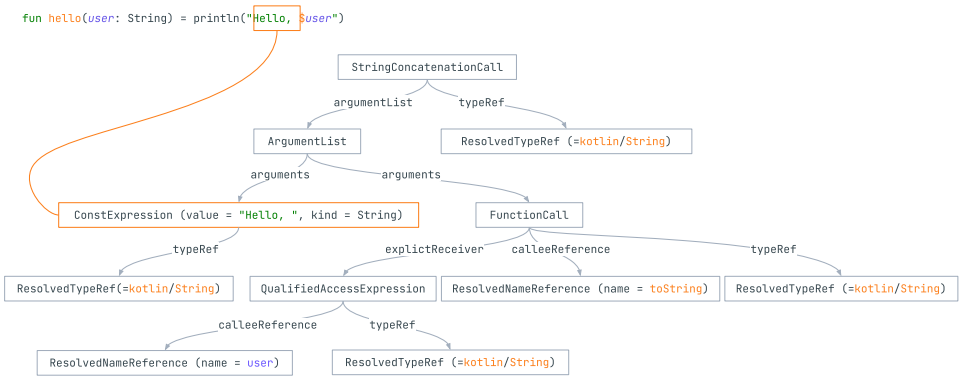
FIR? Another tree!



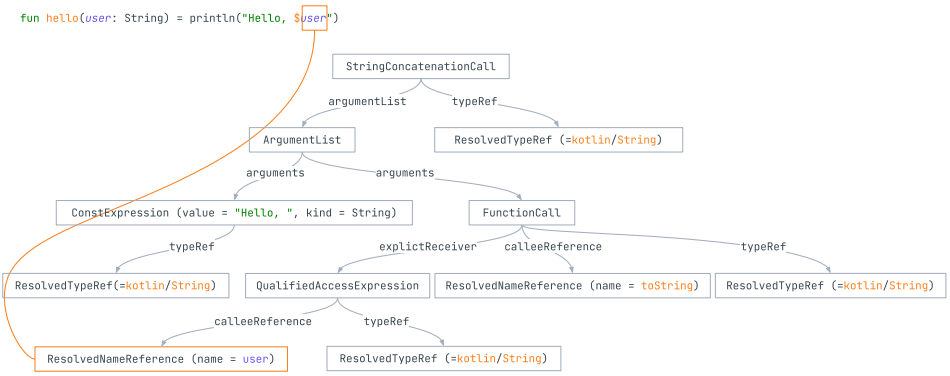
FIR? Another tree!



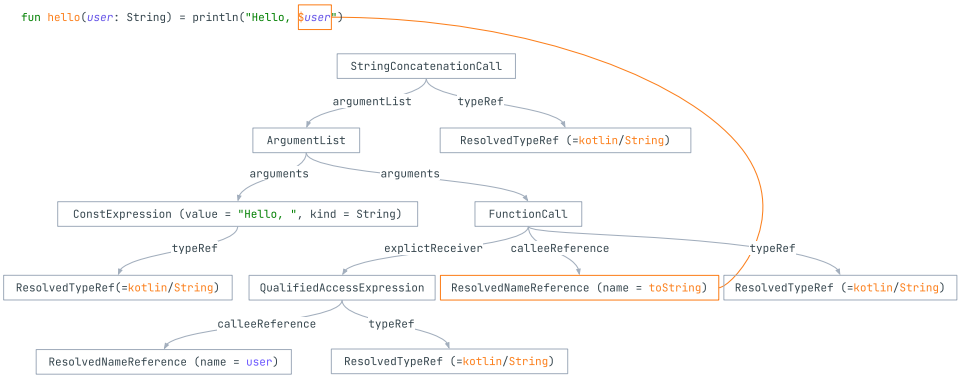
FIR? Another tree!



FIR? Another tree!



FIR? Another tree!

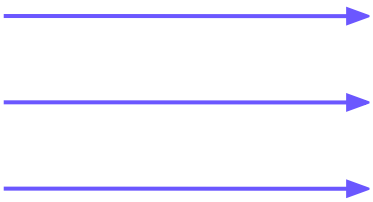


FIR: desugaring

```
if (b) {
    println("Hello")
}

for (s in list) {
    println(s)
}

val (a, b) = "a" to "b"
```

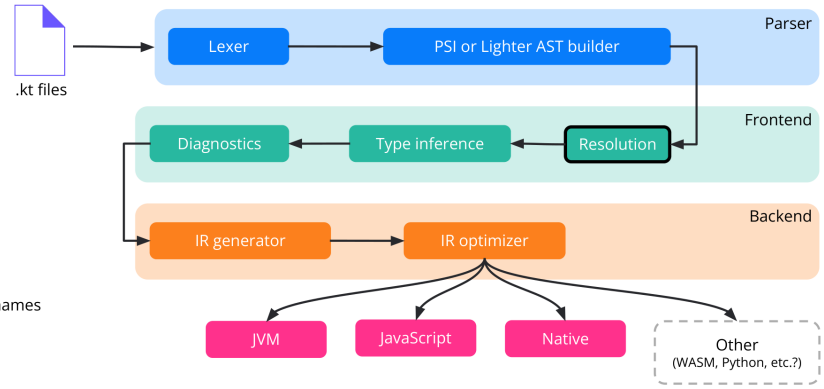


```
when {
    b -> println("Hello")
}

val <iterator> = list.iterator()
while (<iterator>.hasNext()) {
    val s = <iterator>.next()
    println(s)
}

val <destruct> = "a" to "b"
val a = <destruct>.component1()
val b = <destruct>.component2()
```

Kotlin compiler



Resolving the FQ names

Kotlin compiler: resolve

```
fun myFunction() {
}
```

Library A

Short FQ name: `myFunction`

Resolved FQ name: `org.libraryA.myFunction`

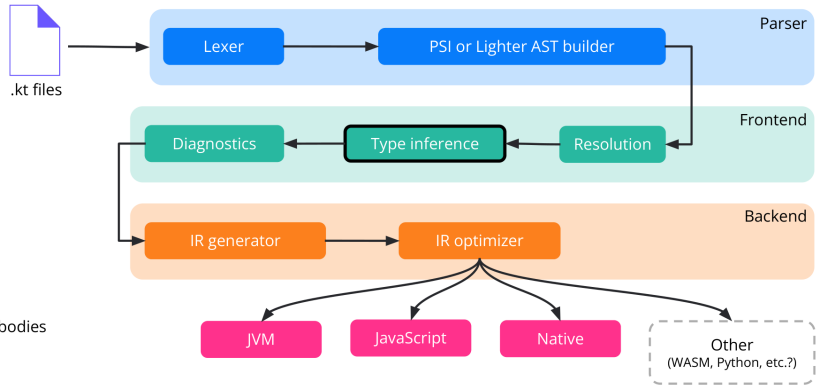
```
fun myFunction() {
}
```

Library B

Short FQ name: `myFunction`

Resolved FQ name: `org.libraryB.myFunction`

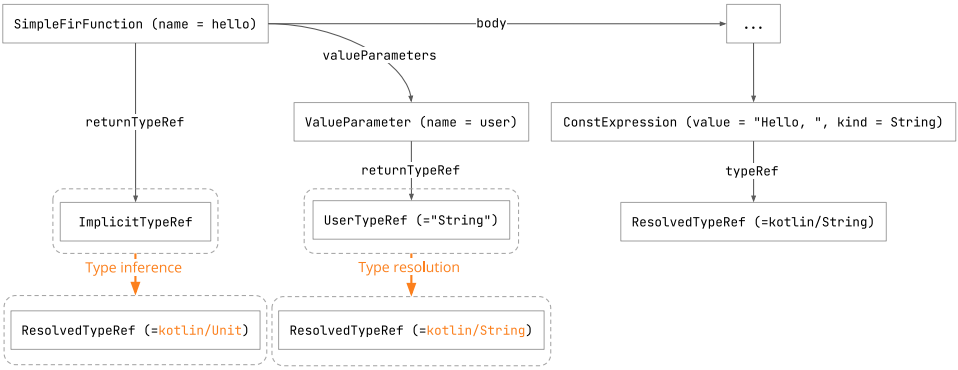
Kotlin compiler



Infers all types and
resolves functions bodies

Kotlin compiler

```
fun hello(user: String) = println("Hello, $user")
```



Java interoperability: nullability

- Java nullable types in Kotlin

Java sources

```java
public class Main {
    public static String foo() {
        // TODO
    }
}
```


Kotlin sources

```kotlin
var a: ???? = foo()
```

Java interoperability: nullability

- Java nullable types in Kotlin

Java sources

```java
public class Main {
    public static String foo() {
        // TODO
    }
}
```


Kotlin sources

```
var a: ???? = foo()

String!
```

Java interoperability: nullability

- Java nullable types in Kotlin
- `String!` is the type range: `[String..String?]`

Java sources

```java
public class Main {
    public static String foo() {
        // TODO
    }
}
```


Kotlin sources

```kotlin
var a: ???? = foo()

String!
```

Java interoperability: nullability

- Java nullable types in Kotlin
- Nullability annotations `@NotNull` and `@Nullable`

Java sources

```java
public class Main {
    @NotNull
    public static String foo() {
       // TODO
    }
}
```


Kotlin sources

```kotlin
var a: String = foo()
```

Java interoperability: collection mapping

- Java collection types in Kotlin

Java sources

```java
public class Main {
    @NotNull
    public static List<@NotNull String> foo() {
        // TODO
    }
}
```


Kotlin sources

```kotlin
var a: ???? = foo()
```

Java interoperability: collection mapping

- Java collection types in Kotlin
- `(Mutable)List<T>` is the type range: `[MutableList<T>..List<T>]`

Java sources

```java
public class Main {
    @NotNull
    public static List<@NotNull String> foo() {
        // TODO
    }
}
```


Kotlin sources

```kotlin
var a: ???? = foo()
```

`(Mutable)List<String>`

Kotlin compiler: control- and data-flow analysis

- Variable initialization analysis
- Return analysis
- Smart cast analysis

Each variable is initialized before being used.

Each immutable variable is not reassigned after initialization.

```
val a: Int

while(true) {
   if (Random.nextBoolean()) {
       a = 15
       break
   }
}

println(a) // It compiles!
```

Kotlin compiler: control- and data-flow analysis

- Variable initialization analysis
- Return analysis
- Smart cast analysis

If the return type is not `Unit`, then the function won't return control unless it returns something.

```
fun bar(): Int {
    print("Again")
    while (true) {
        print(" and again")
    }
} // It compiles!

fun baz(): Long {
    error("YOLO! :)")
} // It compiles!
```

Kotlin compiler: control- and data-flow analysis

- Variable initialization analysis
- Return analysis
- Smart cast analysis

If type check is successful, then the checked value is automatically casted to the corresponding type.

```
fun Any?.printFirstElement() {
    when (this) {
        is List<*> → get(0)
        is Iterable<*> → iterator().next()
    }
}

fun String?.length(): Int =
    if (this == null) 0
    else length

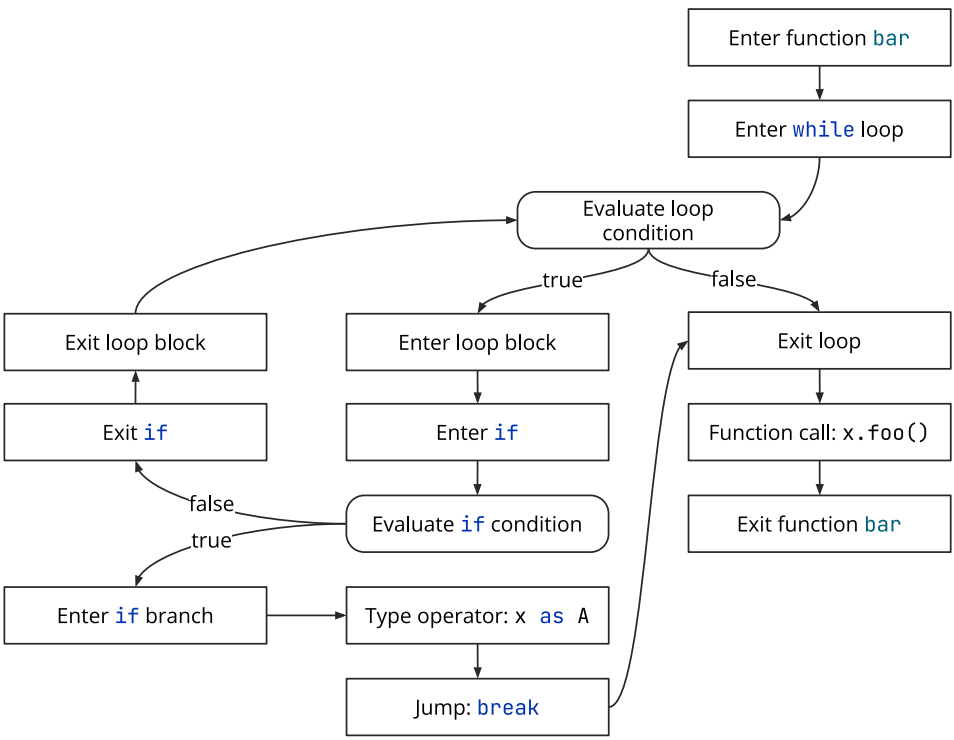
fun Int?.isEven(): Boolean =
    this ≠ null && this % 2 == 0
```

Kotlin compiler: control- and data-flow analysis

```
interface A {
    fun foo()
}
```

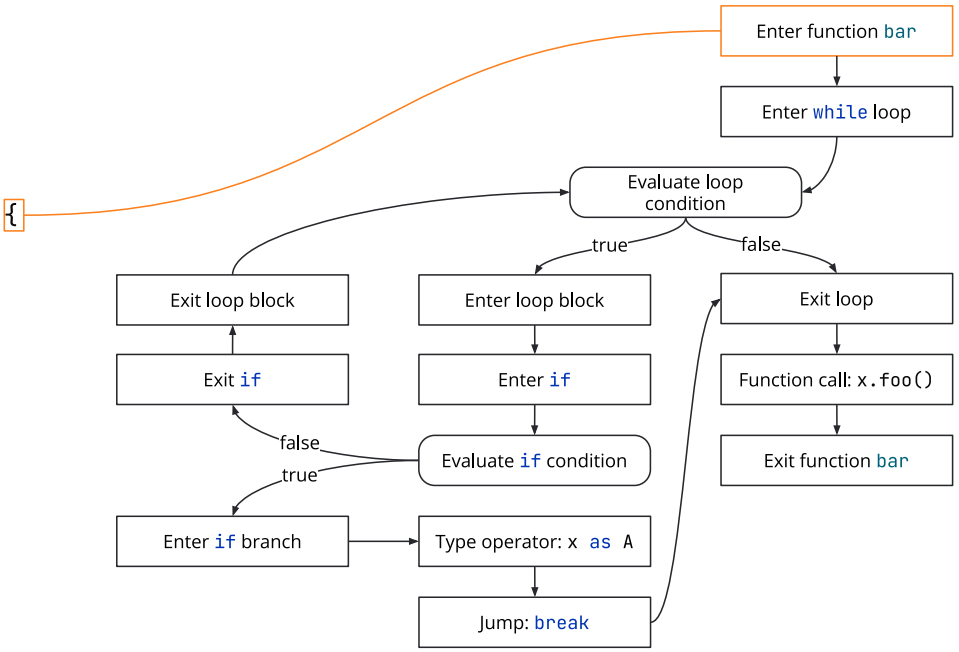
```
                    }
                }
                x.foo()
            }
```

```
fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
```



Kotlin compiler: control- and data-flow analysis

```
        }
    }
    x.foo()
}
```



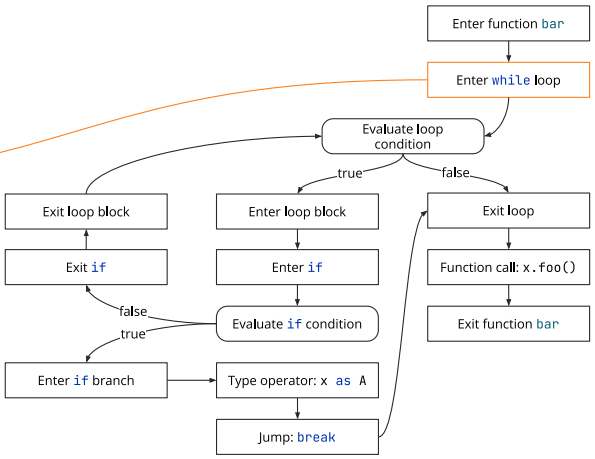

```
interface A {
    fun foo()
}
```

```
fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
```

Kotlin compiler: control- and data-flow analysis

```
interface A {
    fun foo()
}

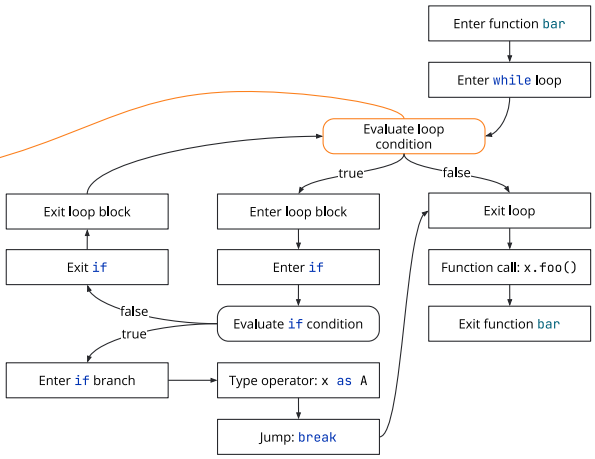
fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



Kotlin compiler: control- and data-flow analysis

```kotlin
interface A {
    fun foo()
}

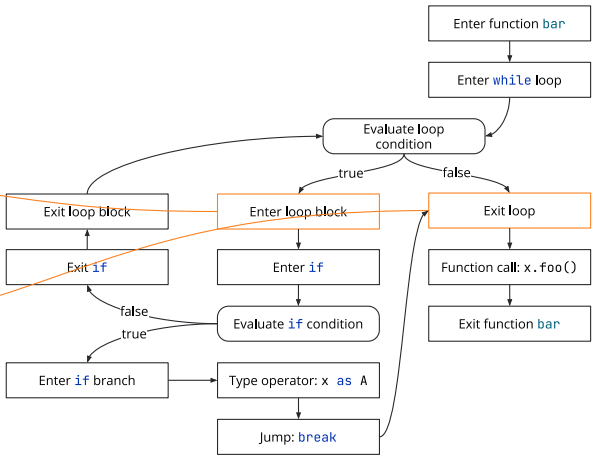
fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



Kotlin compiler: control- and data-flow analysis

```
interface A {
   fun foo()
}

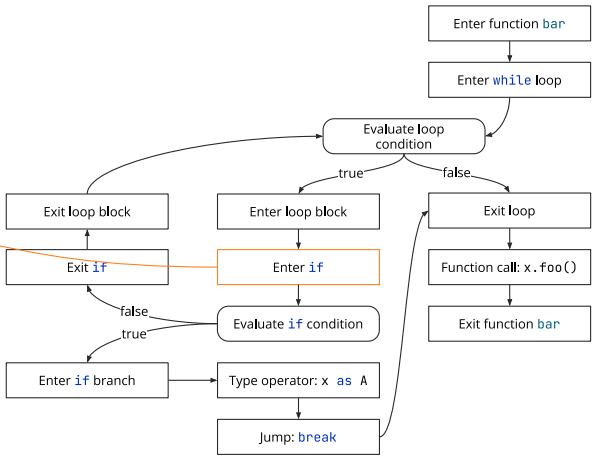
fun bar(x: Any, b: Boolean) {
   while (true) {
       if (b) {
           x as A
           break
       }
   }
   x.foo()
}
```



Kotlin compiler: control- and data-flow analysis

```kotlin
interface A {
    fun foo()
}

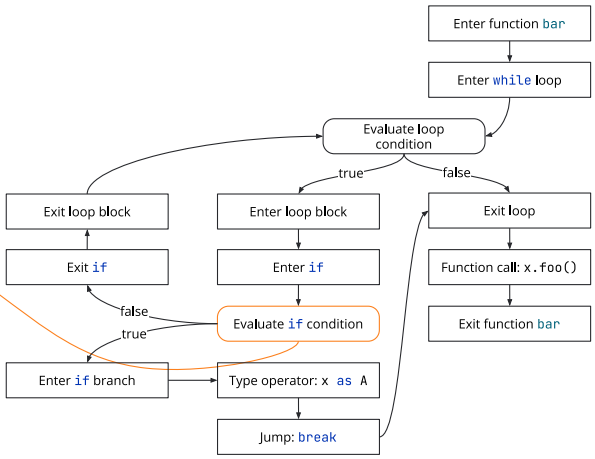
fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



Kotlin compiler: control- and data-flow analysis

```kotlin
interface A {
    fun foo()
}

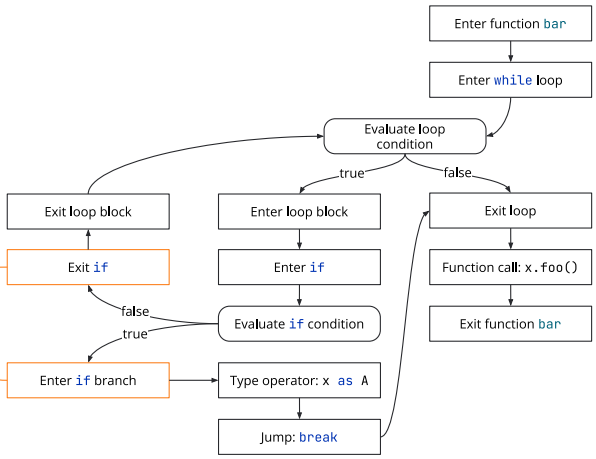
fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



Kotlin compiler: control- and data-flow analysis

```kotlin
interface A {
    fun foo()
}

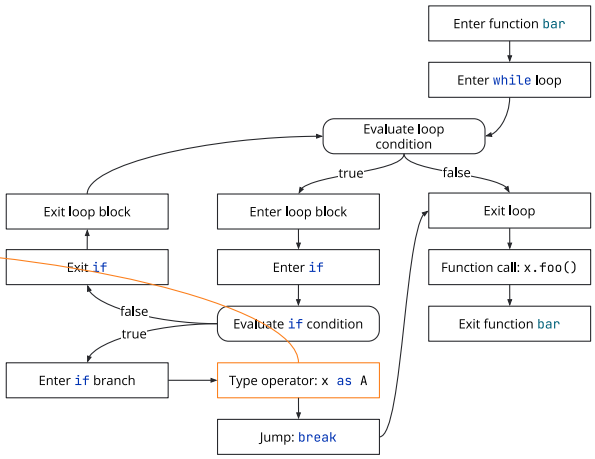
fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



Kotlin compiler: control- and data-flow analysis

```
interface A {
   fun foo()
}

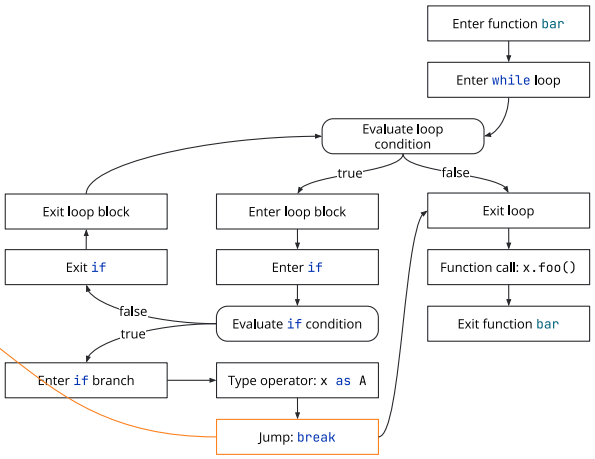
fun bar(x: Any, b: Boolean) {
   while (true) {
       if (b) {
           x as A
           break
       }
   }
   x.foo()
}
```



Kotlin compiler: control- and data-flow analysis

```
interface A {
   fun foo()
}

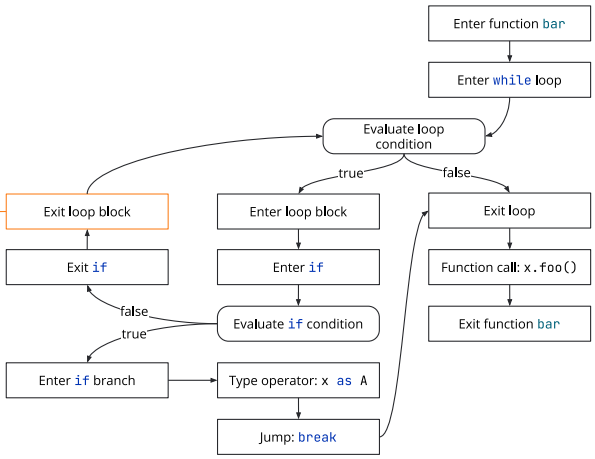
fun bar(x: Any, b: Boolean) {
   while (true) {
       if (b) {
           x as A
           break
       }
   }
   x.foo()
}
```



Kotlin compiler: control- and data-flow analysis

```
interface A {
   fun foo()
}

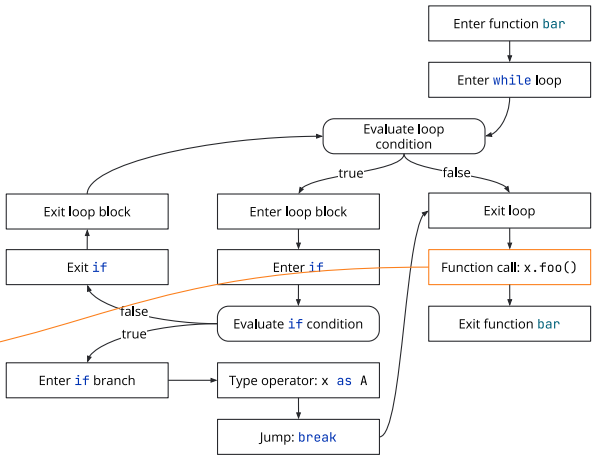
fun bar(x: Any, b: Boolean) {
   while (true) {
       if (b) {
           x as A
           break
       }
   }
   x.foo()
}
```



Kotlin compiler: control- and data-flow analysis

```
interface A {
   fun foo()
}

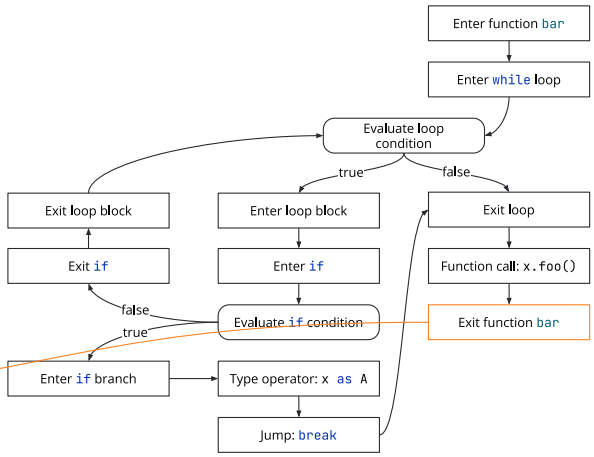
fun bar(x: Any, b: Boolean) {
   while (true) {
       if (b) {
           x as A
           break
       }
   }
   x.foo()
}
```



Kotlin compiler: control- and data-flow analysis

```
interface A {
    fun foo()
}

fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



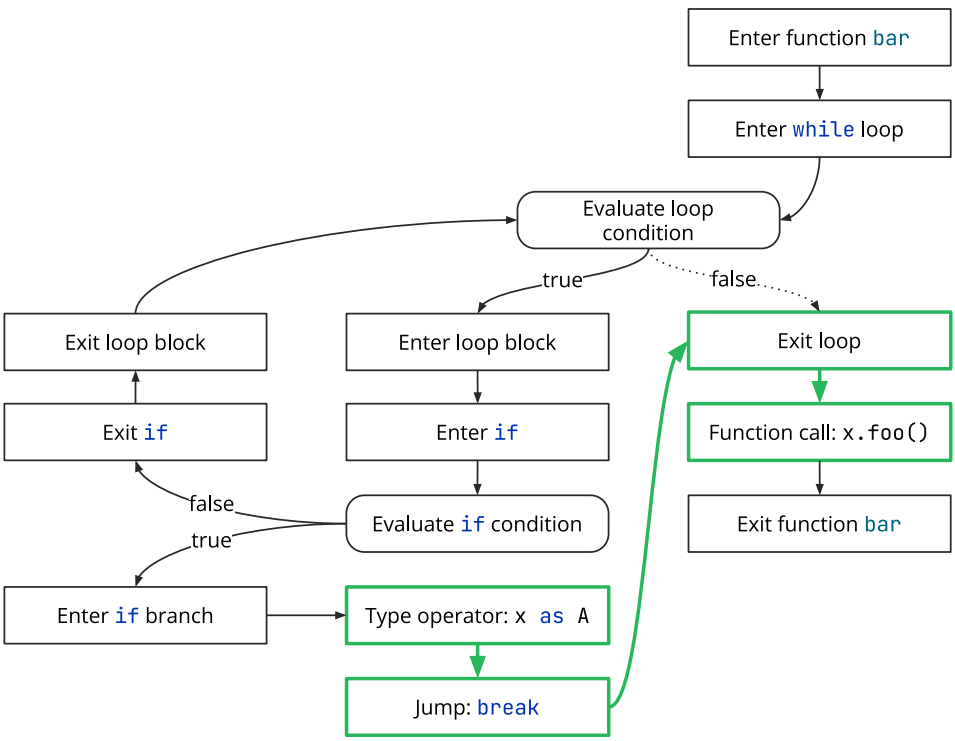
Kotlin compiler: control- and data-flow analysis

```
interface A {
    fun foo()
}
```

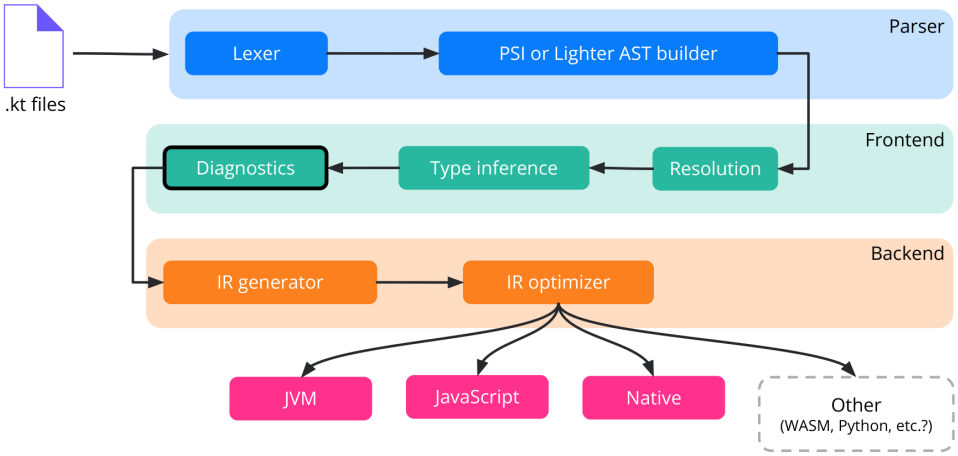
```
            }
        }
        x.foo()
    }
```

x is A

```
fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
```



Kotlin compiler



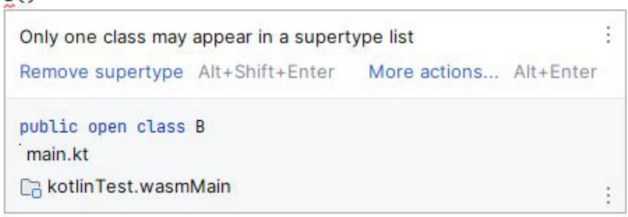
Almost all checks

you've ever seen in

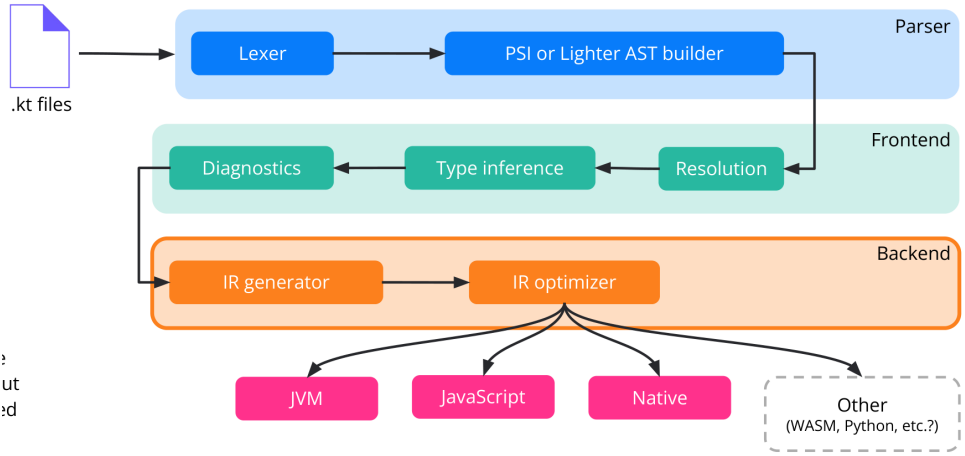
IntelliJ IDEA.

Kotlin compiler: diagnostics

```
open class A
open class B
open class C: A(), B()
```

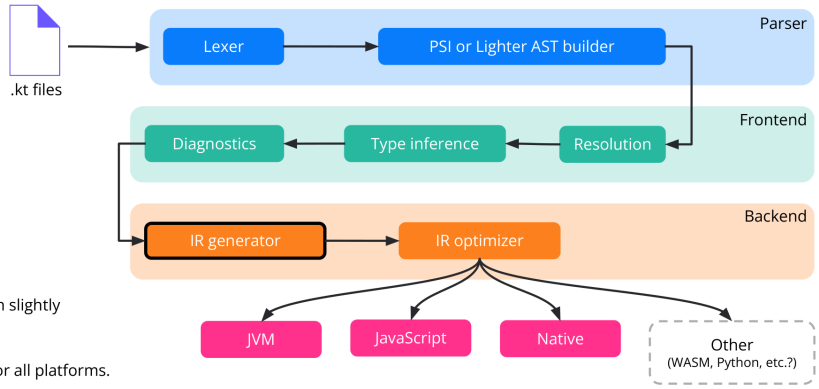


Kotlin compiler



On the backend, we **DO NOT resolve**, but only use the received information

Kotlin compiler



IR, a representation slightly different from FIR.

It is still common for all platforms.

Kotlin compiler: resolved tree

```
fun hello(user: String) = println("Hello,
$user")
```

Kotlin compiler: resolved tree

```
public fun hello(user: kotlin.String): kotlin.Unit
    defined in example in file Example.kt
```

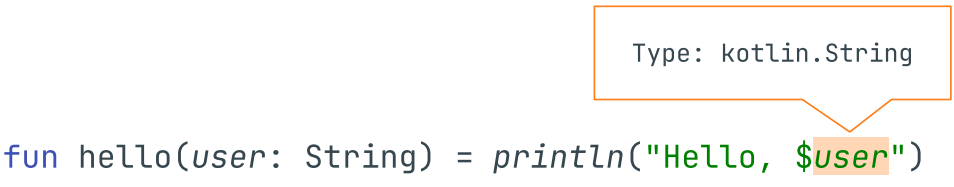
```
fun hello(user: String) = println("Hello, $user")
```

Kotlin compiler: resolved tree

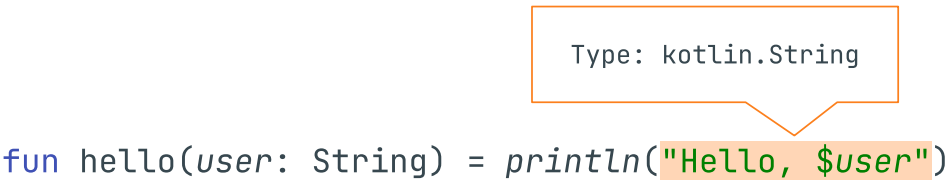
Reference to *value-parameter* *user*: kotlin.String
defined in example.hello

```
fun hello(user: String) = println("Hello,
$user")
```

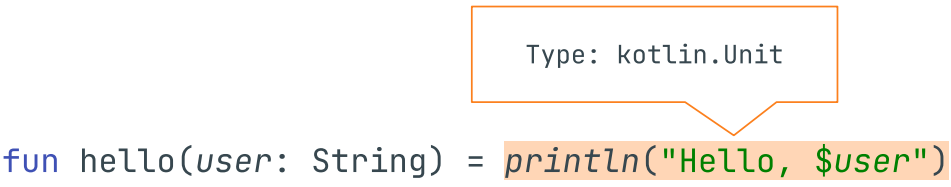
Kotlin compiler: resolved tree



Kotlin compiler: resolved tree



Kotlin compiler: resolved tree



Kotlin compiler: resolved tree




```
fun hello(user: String) = println("Hello,
$user")
```

IR? Yet another tree!

The code:

```
// file 'src/kotlin/example.kt'

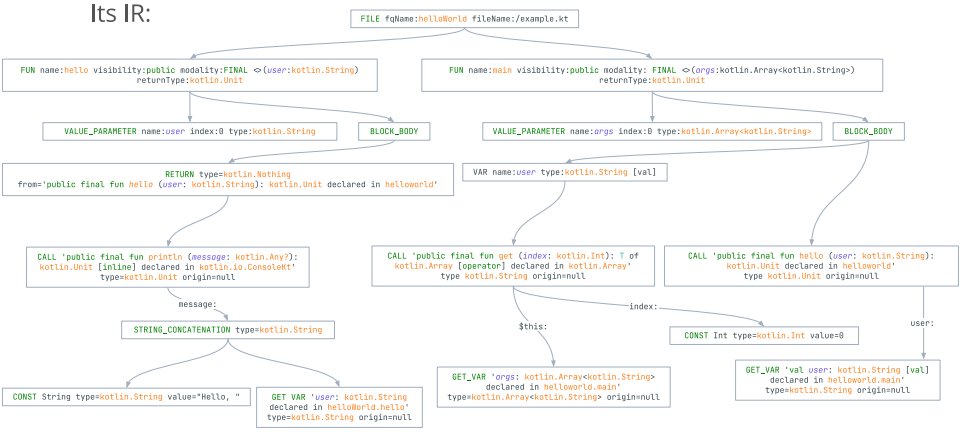
package helloWorld

fun hello(user: String) = println("Hello,
$user")

fun main(args: Array<String>) {
    val user = args[0]
    hello(user)
}
```

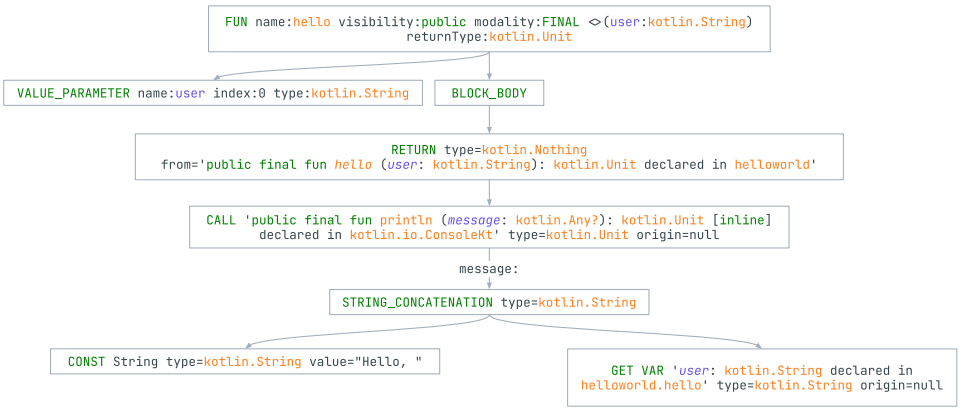
IR? Yet another tree!

Its IR:

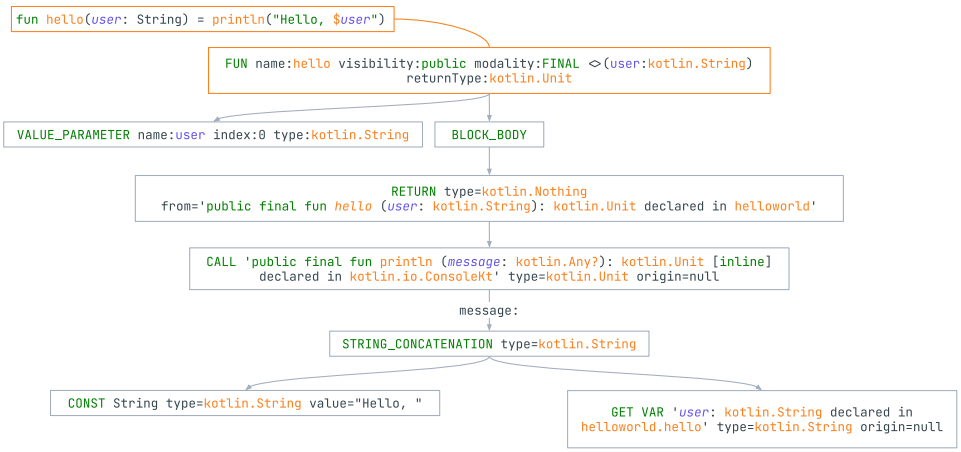


Back-end intermediate representation: a closer look

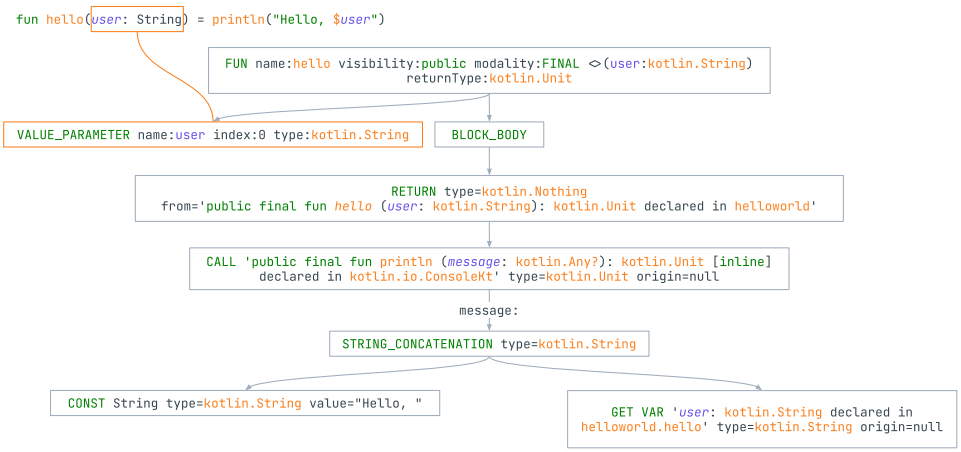
```
fun hello(user: String) = println("Hello, $user")
```



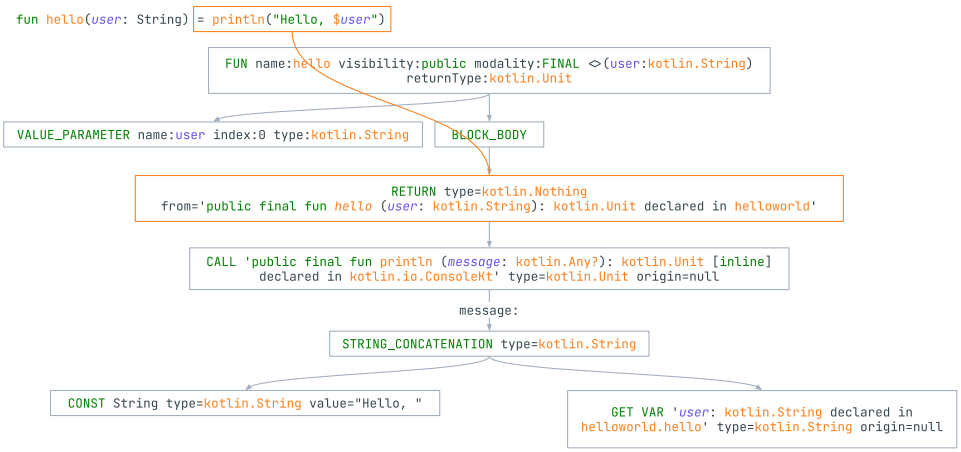
Back-end intermediate representation: a closer look



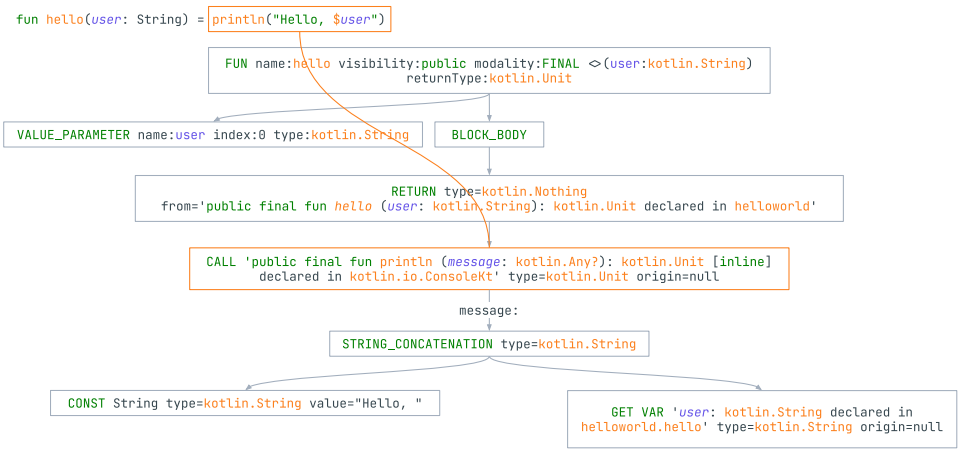
Back-end intermediate representation: a closer look



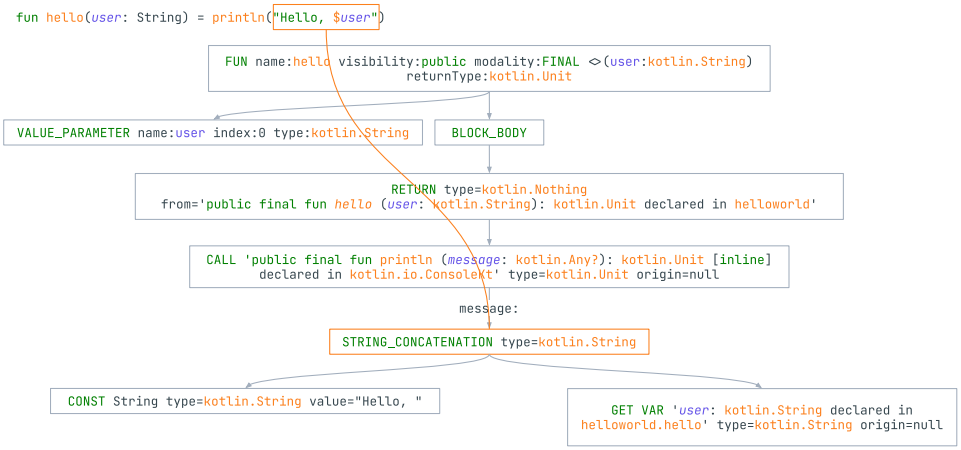
Back-end intermediate representation: a closer look



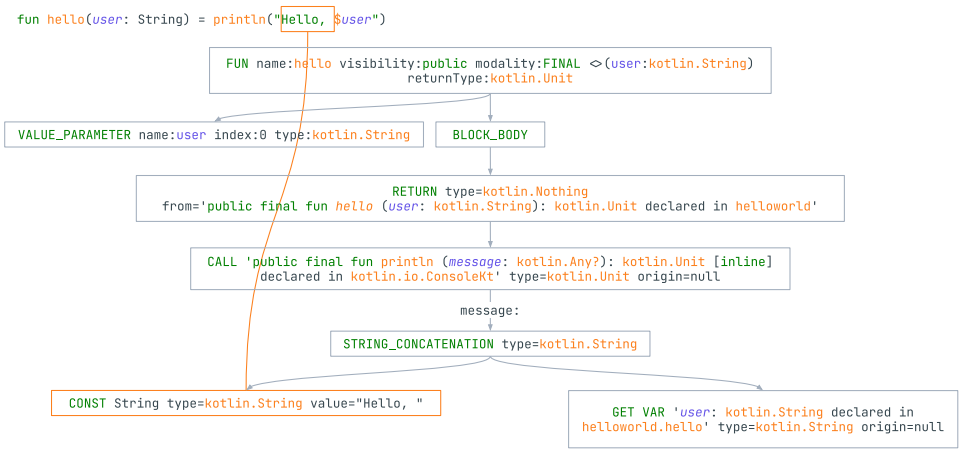
Back-end intermediate representation: a closer look



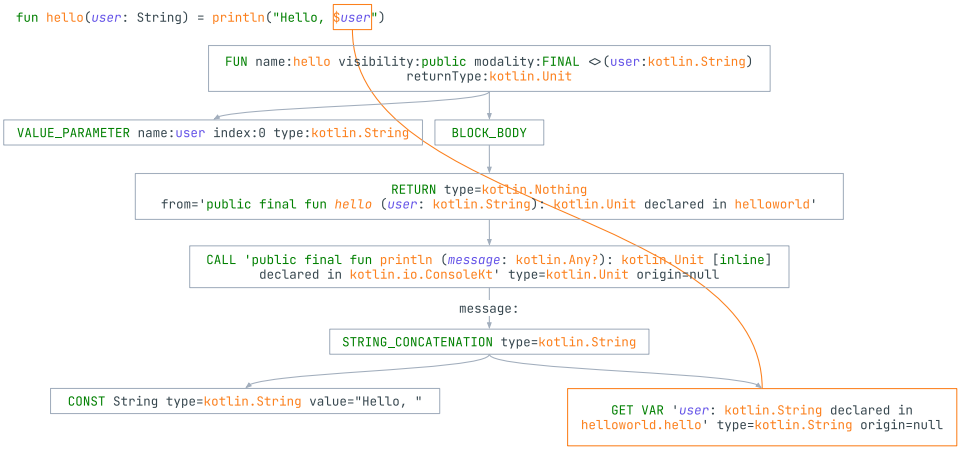
Back-end intermediate representation: a closer look



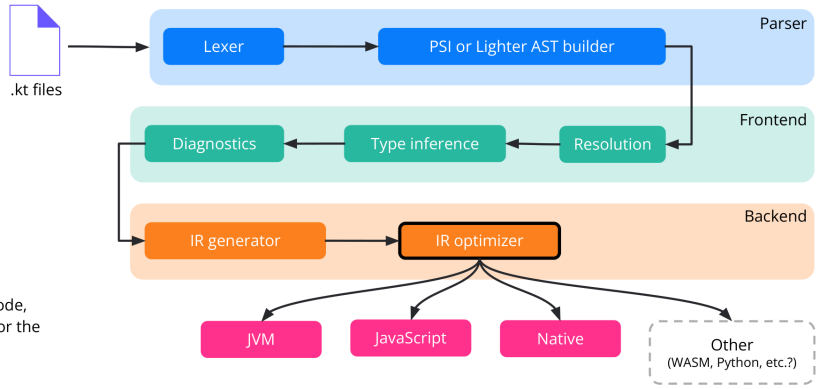
Back-end intermediate representation: a closer look



Back-end intermediate representation: a closer look

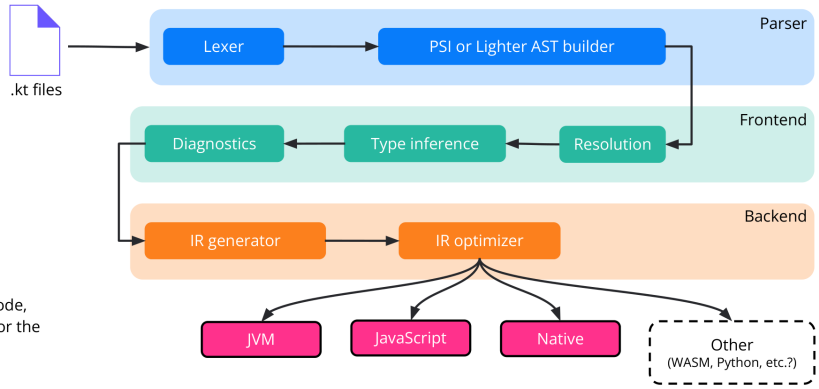


Kotlin compiler



Platform-specific code, such as bytecode for the JVM

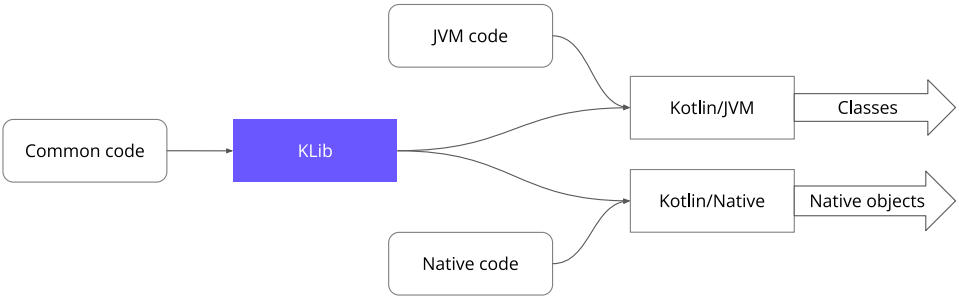
Kotlin compiler



Platform-specific code, such as bytecode for the JVM

KLibs

JAR analogues – store a serialized IR for the subsequent use of cross-platform libraries.



Compiler plugins

You can extend **any** compile phase via compiler plugins. To do so, you need to implement compiler extensions and register them.

To register extensions, you need to inherit from `CompilerPluginRegistrar` (`ComponentRegistrar` for previous versions).

Don’t forget to use `supportsK2 = true` if you need to support the FIR frontend

Compiler plugins: FIR extensions

To get the full list of the extensions, please see: `package org.jetbrains.kotlin.fir.extensions` (link).

Consider an example:

```
/*
 * Generates top level class
 *
 * package foo.bar
 *
 * public final class MyClass {
 *     fun foo(): String = "Hello world"
 * }
 */
```

Compiler plugins: FIR extensions

```
class SimpleClassGenerator(session: FirSession) : FirDeclarationGenerationExtension(session) {
    companion object {
          // foo.bar.MyClass
        val MY_CLASS_ID = ClassId(
            FqName.fromSegments(listOf("foo", "bar")),
            Name.identifier("MyClass")
        )
          // foo.bar.MyClass.foo
        val FOO_ID = CallableId(MY_CLASS_ID, Name.identifier("foo"))
    }
    override fun generateClassLikeDeclaration(classId: ClassId): FirClassLikeSymbol<*>? {
        ...
    }
    ...
}
```

Compiler plugins: FIR extensions

```
class SimpleClassGenerator(session: FirSession) : FirDeclarationGenerationExtension(session) {
    companion object {
          // foo.bar.MyClass
        val MY_CLASS_ID = ClassId(
            FqName.fromSegments(listOf("foo", "bar")),
            Name.identifier("MyClass")
        )
          // foo.bar.MyClass.foo
        val FOO_ID = CallableId(MY_CLASS_ID, Name.identifier("foo"))
    }
    override fun generateClassLikeDeclaration(classId: ClassId): FirClassLikeSymbol<*>? {
        ...
    }
    ...
}
```

Compiler plugins: FIR extensions

```
class SimpleClassGenerator(session: FirSession) : FirDeclarationGenerationExtension(session) {
    companion object {
          // foo.bar.MyClass
        val MY_CLASS_ID = ClassId(
            FqName.fromSegments(listOf("foo", "bar")),
            Name.identifier("MyClass")
        )
          // foo.bar.MyClass.foo
        val FOO_ID = CallableId(MY_CLASS_ID, Name.identifier("foo"))
    }
    override fun generateClassLikeDeclaration(classId: ClassId): FirClassLikeSymbol<*>? {
        ...
    }
    ...
}
```

Compiler plugins: FIR extensions

```
class SimpleClassGenerator(session: FirSession) : FirDeclarationGenerationExtension(session) {
    companion object {
          // foo.bar.MyClass
        val MY_CLASS_ID = ClassId(
            FqName.fromSegments(listOf("foo", "bar")),
            Name.identifier("MyClass")
        )
          // foo.bar.MyClass.foo
        val FOO_ID = CallableId(MY_CLASS_ID, Name.identifier("foo"))
    }
    override fun generateClassLikeDeclaration(classId: ClassId): FirClassLikeSymbol<*>? {
        ...
    }
    ...
}
```

Compiler plugins: FIR extensions

```
class SimpleClassGenerator(session: FirSession) : FirDeclarationGenerationExtension(session) {
    companion object {
          // foo.bar.MyClass
        val MY_CLASS_ID = ClassId(
            FqName.fromSegments(listOf("foo", "bar")),
            Name.identifier("MyClass")
        )
          // foo.bar.MyClass.foo
        val FOO_ID = CallableId(MY_CLASS_ID, Name.identifier("foo"))
    }
    override fun generateClassLikeDeclaration(classId: ClassId): FirClassLikeSymbol<*>? {
        ...
    }
    ...
}
```

Compiler plugins: FIR extensions

You use a special key to mark **everything** generated by the compiler and can transfer **any**

information between frontend and backend. So it also helps to find the new declaration to generate

it's IR.

```
class SimpleClassGenerator(session: FirSession) :
FirDeclarationGenerationExtension(session) {

        object Key : GeneratedDeclarationKey()

...

}
```

Compiler plugins: FIR extensions

Don’t forget to register all new declarations.

```
    ...
    override fun getTopLevelClassIds(): Set<ClassId> {
        return setOf(MY_CLASS_ID)
    }
```

```
class SimpleClassGenerator(session: FirSession) :
FirDeclarationGenerationExtension(session) {

    override fun hasPackage(packageFqName: FqName): Boolean {
        return packageFqName == MY_CLASS_ID.packageFqName
    }

}
```

Compiler plugins: IR extensions

Actually, the compiler has only one extension for IRs: `IrGenerationExtension`.

You just need to implement some *transformers* and accept them:

```
class SimpleIrGenerationExtension: IrGenerationExtension {
    override fun generate(moduleFragment: IrModuleFragment,
pluginContext: IrPluginContext) {
        val transformers =
listOf(SimpleIrBodyGenerator(pluginContext))
        for (transformer in transformers) {
            moduleFragment.acceptChildrenVoid(transformer)
        }
    }
}
```

Don’t forget to check the `key` (use the `interestedIn` function)!

Compiler plugins: popular plugins

- kotlinx.serialization — Generates visitor code for serializable classes.
- all-open — Marks all classes as open classes.
- kapt — An annotation processor.
- ksp — An API for developing lightweight compiler plugins.
- Jetpack Compose — Generates efficient UI from its declarative description.
- Arrow Meta — Compiler plugin API that empowers all Arrow libraries.
- ...

JETBRAINS | Kotlin

Thank you

